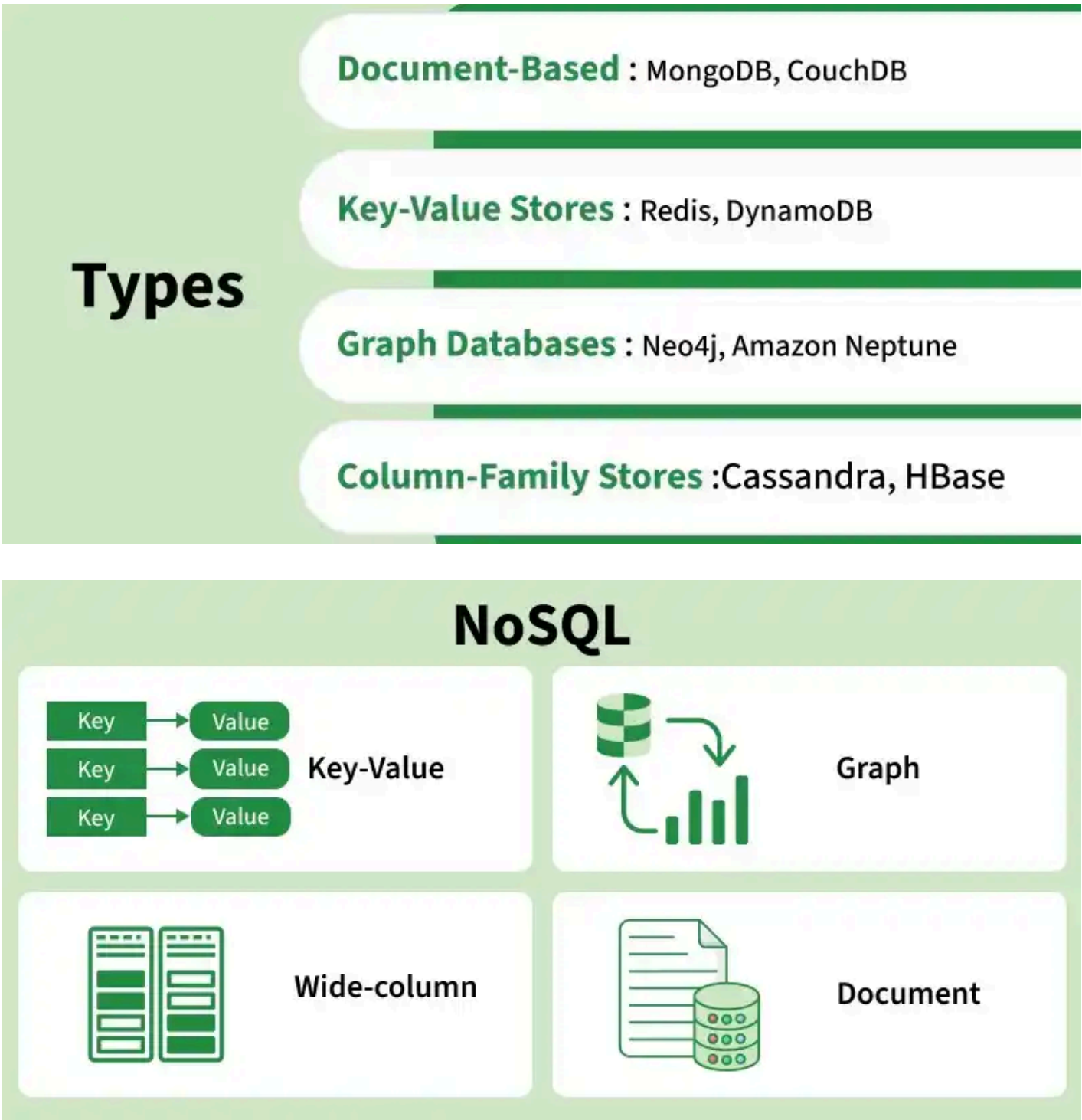


Chapter 1 Basics of NoSQL

1. Introduction to NoSQL

A NoSQL database is a type of database that stores data using non-relational data models instead of traditional tables.

- Designed to efficiently manage large volumes of structured, semi-structured and unstructured data.
- Work with cloud platforms and distributed systems using tools such as Docker, Kubernetes and Apache Cassandra.
- Optimizes performance by reducing complex JOIN operations.
- Used by high-traffic applications such as social media, e-commerce, gaming and real-time analytics to handle massive volumes of data and concurrent users.



Example

An e-commerce application stores product information where different products have different attributes.

Traditional SQL Table

ProductID	Name	Color	Size	RAM	Storage
101	T-Shirt	Blue	M	NULL	NULL
102	Smartphone	NULL	NULL	8 GB	128 GB

Many fields remain empty because all products share the same table.

NoSQL Document

```
{
  "ProductID": 101,
  "Name": "T-Shirt",
  "Color": "Blue",
  "Size": "M"
}
```

```
{
  "ProductID": 102,
  "Name": "Smartphone",
  "RAM": "8 GB",
  "Storage": "128 GB"
}
```

Each document stores only the required fields, making the database more flexible.

1.1 Features

- **Horizontal Scalability:** Distributes data across multiple nodes, enabling the database to handle increasing workloads by adding more servers.
- **Distributed Data Storage:** Stores and manages data across a cluster of machines, improving availability and supporting large-scale deployments.
- **Multiple Data Models:** Supports document, key-value, column-family and graph models, allowing developers to choose the most suitable model for different use cases.
- **High Availability:** Uses replication and automatic failover mechanisms to ensure continuous access to data even when individual nodes become unavailable.
- **Fault Tolerance:** Detects node failures and continues serving requests using replicated data and distributed storage.

1.2 Popular NoSQL Databases

- MongoDB
- Apache Cassandra
- Redis
- Apache HBase
- Neo4j
- CouchDB
- Couchbase
- Amazon DynamoDB
- Azure Cosmos DB
- Google Cloud Bigtable

1.3 Applications

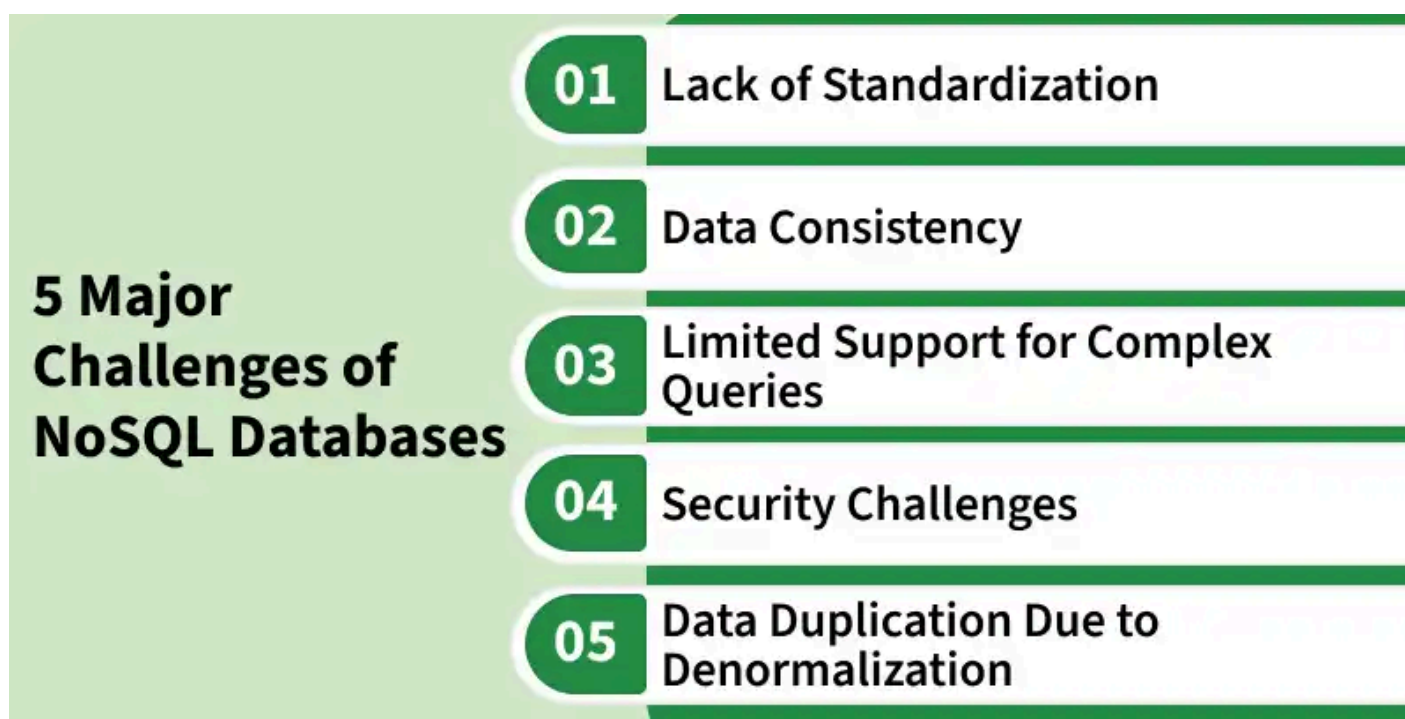
- **Content Management Systems (CMS):** Efficiently stores and retrieves articles, multimedia files and metadata with varying structures.
- **Internet of Things (IoT):** Manages high-frequency sensor and device data generated by connected systems.
- **Caching and Session Management:** Provides low-latency storage for user sessions, authentication tokens and frequently accessed data.
- **Big Data Processing:** Stores and processes massive datasets generated from distributed applications and analytics platforms.

1.4 Challenges

- **Lack of standardization:** Different databases use different query languages.
- **Limited complex queries:** Joins and complex operations are less efficient.
- **Management complexity:** Distributed systems can be difficult to manage.

NoSQL databases are widely used for handling large and unstructured data because of their flexibility and scalability. However, they also have certain limitations and challenges compared to relational databases.

- They may lack strong consistency compared to relational databases.
- Standardization is limited because different NoSQL databases use different models.
- Complex queries and joins are harder to perform.
- Data duplication may occur due to denormalization.



1.4.1 Lack of Standardization

Different NoSQL databases use different data models, architectures and query languages. Unlike relational databases that commonly use SQL, NoSQL systems do not follow a single standard query language.

- Different databases use different syntaxes and structures.
- Developers must learn separate technologies for each database.
- Switching from one NoSQL database to another can be difficult.

Examples:

- MongoDB uses a document-based model and MongoDB Query Language (MQL).
- Apache Cassandra uses a wide-column model and Cassandra Query Language (CQL).
- Redis mainly works as a key-value database.

1.4.2 Data Consistency

Many NoSQL databases prioritize scalability and availability over strong consistency. As a result, they often follow the principle of eventual consistency instead of immediate consistency.

- Data updates may not appear instantly across all servers.
- Temporary inconsistencies can occur in distributed systems.
- Synchronizing data across multiple nodes becomes challenging.

Examples:

- Apache Cassandra commonly follows eventual consistency in distributed environments.
- Older versions of MongoDB had limited multi-document transaction support compared to relational databases.

1.4.3 Limited Support for Complex Queries

Some NoSQL databases are not optimized for advanced querying operations and relational data handling.

- Limited support for joins and relationships.
- Complex queries may require additional application-side logic.
- Performing multi-table operations efficiently can be difficult.

Examples:

- Redis is highly optimized for fast key-value access but not for complex relational queries.
- MongoDB supports aggregation pipelines, but joins are still more limited compared to SQL databases.

1.4.4 Security Challenges

Managing security in NoSQL databases can become difficult, especially in large-scale distributed environments.

- Some databases provide limited built-in security mechanisms.
- Access control and authentication may require extra configuration.
- Improper configuration can increase security vulnerabilities.
- Monitoring security across distributed systems requires additional effort.

1.4.5 Data Duplication Due to Denormalization

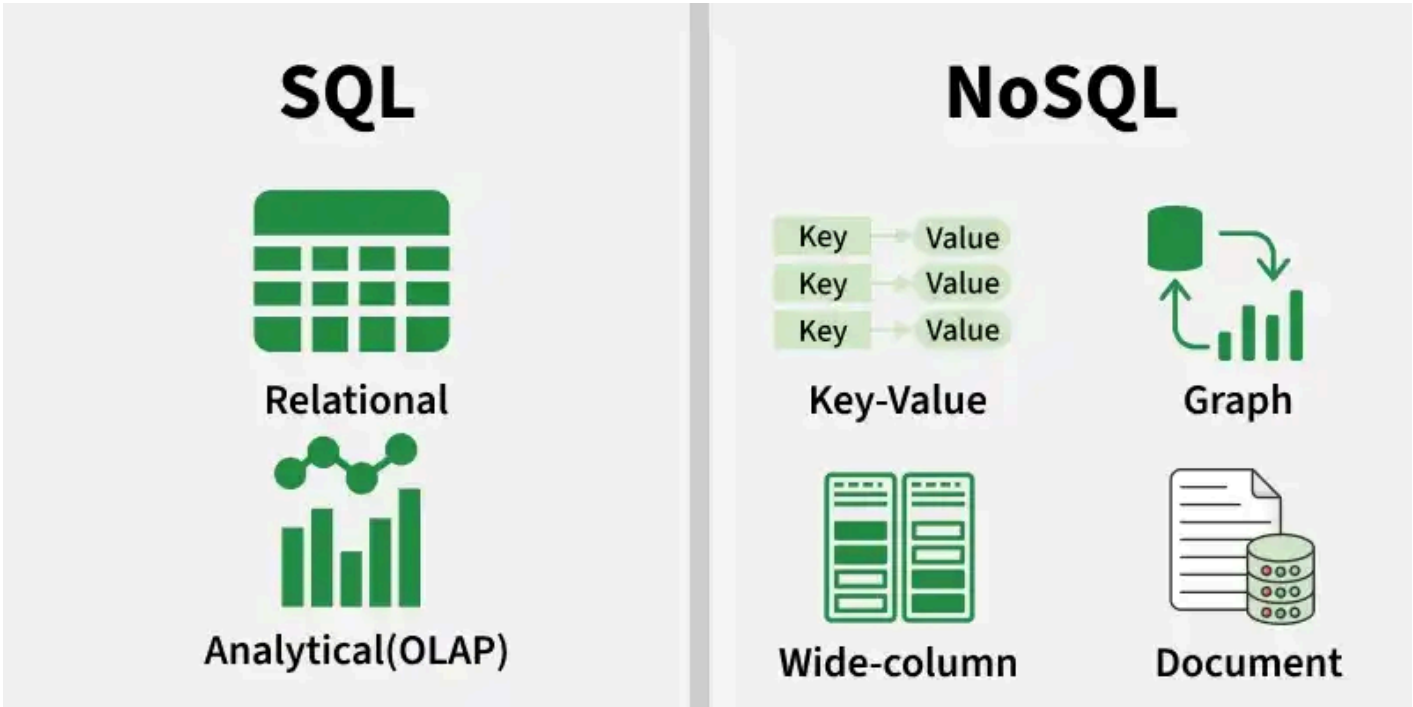
NoSQL databases often use denormalization to improve read performance and scalability. This can lead to data duplication.

- The same data may be stored in multiple locations.
- Storage requirements may increase significantly.
- Updating duplicate data can become difficult.

1.5 SQL vs NoSQL

Databases are used to store, manage and retrieve data efficiently. SQL and NoSQL are the two main types of databases, each designed for different data storage and application requirements.

- **SQL** databases store data in structured tables with predefined schemas and are best suited for complex queries and strong consistency.
- **NoSQL** databases use flexible data models such as document, key-value, graph or column-family and are ideal for large-scale, rapidly changing applications.



The table below shows the major differences between SQL and NoSQL databases:

SQL	NoSQL
Data is stored in tables with rows and columns.	Data is stored as documents, key-value pairs, graphs or columns.
Uses a relational data model.	Uses a non-relational data model.

SQL	NoSQL
Has a fixed schema.	Has a flexible schema.
Supports complex joins between tables.	Generally avoids joins.
Follows ACID properties for transactions.	Usually follows the BASE model.
Scales vertically by increasing server capacity.	Scales horizontally by adding more servers.
Best suited for structured data.	Suitable for structured, semi-structured and unstructured data.
Commonly used in banking, ERP and CRM systems.	Commonly used in social media, Big Data and IoT applications.
Examples: MySQL, PostgreSQL	Examples: MongoDB, Cassandra

The choice between SQL and NoSQL depends on application requirements:

SQL databases are preferred when:

- Data consistency is critical.
- Complex joins and transactions are required.
- The data structure is well-defined.
- Applications involve banking, ERP or inventory management systems.

NoSQL databases are preferred when:

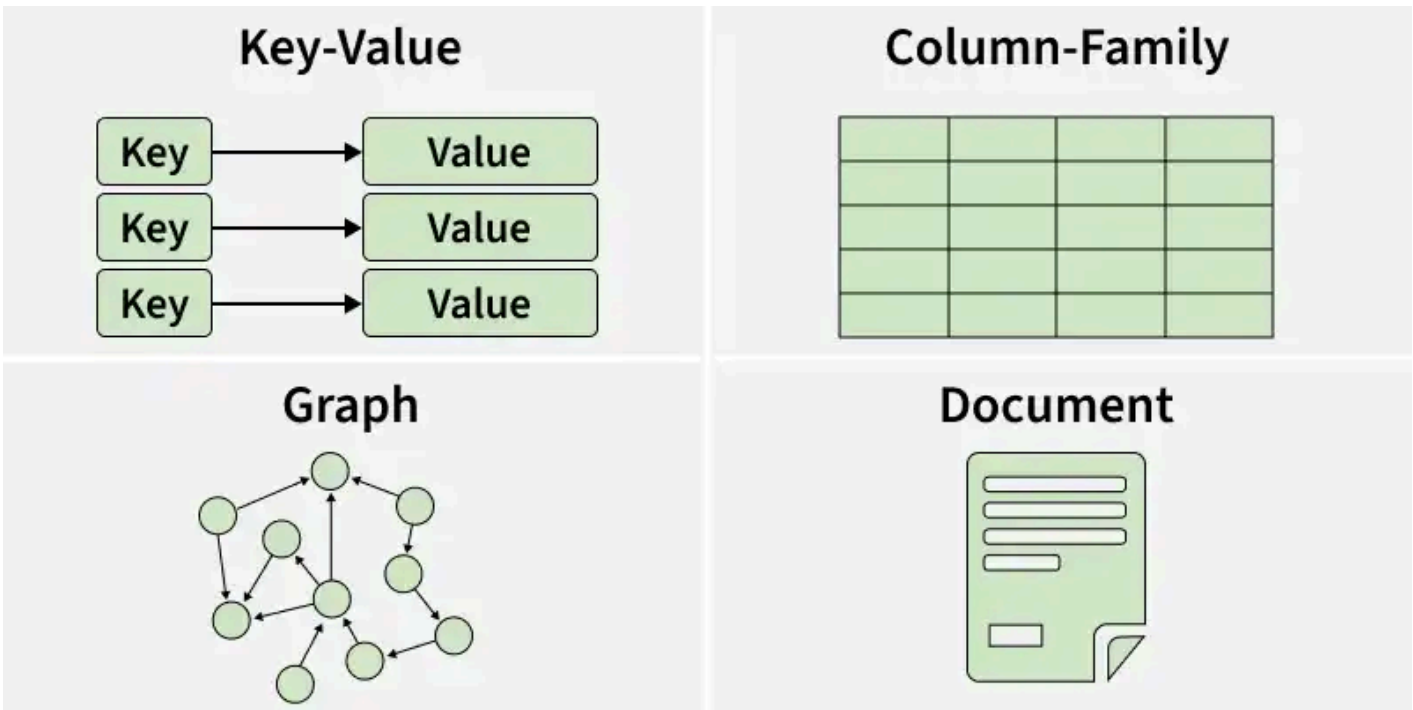
- Applications require high scalability.
- Data is unstructured or semi-structured.
- Real-time processing is required.
- Frequent schema changes are expected.

2. Types of NoSQL Databases

NoSQL databases are classified into different types based on the way they store and organize data. Each type is designed to handle specific data structures and application requirements, offering different advantages in terms of scalability, performance and data access.

There are four main types of NoSQL databases:

- Key-Value Database
- Document Database
- Column-Family Database
- Graph Database



Each type has unique advantages and use cases, making NoSQL a preferred choice for big data applications, real-time analytics, cloud computing and distributed systems.

2.1 Document-Based Database

A **document-based database** is a non-relational database that stores data as documents, offering flexible schemas and efficient access aligned with application data models.

- Stores data in documents (JSON, BSON, or **XML**) instead of tables.
- Closely matches application data objects, reducing data translation.
- Uses collections to group related documents with similar content.
- Allows flexible schemas, so documents in a collection need not be identical.

Popular Document Databases & Use Cases

Database	Use Case
MongoDB	Content management, product catalogs, user profiles
CouchDB	Offline applications, mobile synchronization
Firebase Firestore	Real-time apps, chat applications

2.2 Key-Value Stores

A **key-value store** is a non-relational database that stores data as simple key–value pairs, enabling fast and efficient data access.

- Each key uniquely identifies a value.
- Values can be simple or complex objects.
- Does not support structured schemas, relationships, or SQL-like querying as in relational databases.

Popular Key-Value Databases & Use Cases

Database	Use Case
Redis	Caching, real-time leaderboards, session storage
Memcached	High-speed in-memory caching
Amazon DynamoDB	Cloud-based scalable applications

2.3 Column Oriented Databases

A **column-oriented database** is a non-relational database that stores data by columns rather than rows, enabling efficient analytics and fast data retrieval.

- Read only required columns, reducing memory usage.
- Optimized for read-heavy analytical queries and large-scale data processing.
- Suitable for big data and distributed systems.

Popular Column-Oriented Databases & Use Cases

Database	Use Case
Apache Cassandra	Real-time analytics, IoT applications
Google Bigtable	Large-scale machine learning, time-series data
HBase	Hadoop ecosystem, distributed storage

2.4 Graph-Based Databases

Graph-based databases store data as nodes and relationships, making them ideal for managing and querying complex, interconnected data.

- Used where relationships are important (social networks, fraud detection).
- Supports flexible or optional schema rather than being completely schema-less.
- Efficiently traverses and analyzes connected data using relationships instead of joins.

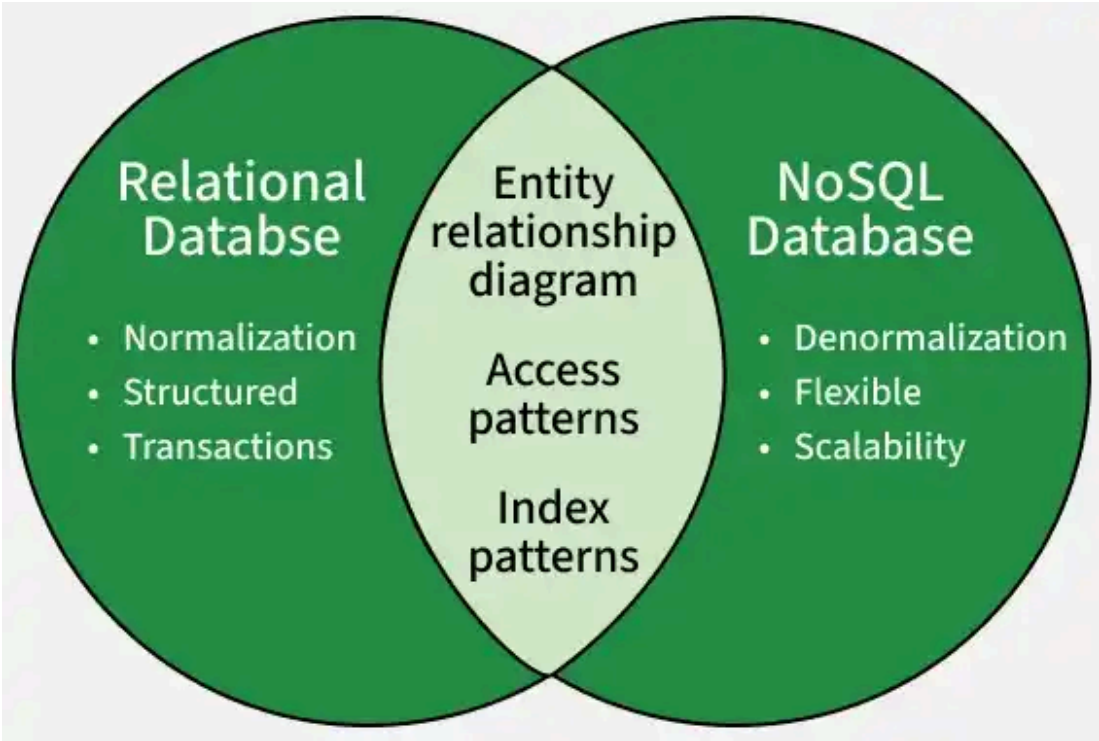
Popular Graph Databases & Use Cases

Database	Use Case
Neo4j	Fraud detection, social networks
Amazon Neptune	Knowledge graphs, AI recommendations
ArangoDB	Multi-model database, cybersecurity

3. Relational Database vs NoSQL

In modern data storage systems, choosing between an RDBMS and a NoSQL database is an important architectural decision. This choice directly impacts application scalability, performance, and development flexibility.

- **RDBMS** emphasises structured data, fixed schemas, and strong consistency
- **NoSQL** focuses on horizontal scalability, flexible schemas, and high performance for large-scale distributed applications.



3.1 Relational Databases vs NoSQL

Feature	Relational (SQL)	NoSQL (Non-Relational)
Data Structure	Structured (Table-based)	Unstructured (Document, Key-Value, Graph)
Schema	Rigid (Defined upfront)	Flexible (Dynamic)
Query Language	SQL (Standardized)	Varies (UnQL, JSON, API-based)
Scaling	Vertical (Add power to server)	Horizontal (Add more servers)
Relationships	Supported via JOINS	Not heavily supported (often denormalized)
Transactions	ACID (Strong consistency)	BASE (Eventual consistency)
Best for	Complex queries, financial systems	High throughput, massive data, agility

3.2 Choosing the Right Database for Your Use Case

Choose RDBMS (SQL) If:

1. **Data Integrity is Critical:** Financial transactions, inventory management, and accounting systems where "eventual" consistency is unacceptable.
2. **Structure is Unlikely to Change:** You have a clear, stable schema.
3. **Complex Relationships:** You need to perform complex JOINS across multiple tables to generate reports.

Choose NoSQL If:

1. **Speed & Scale are Critical:** You need single-digit millisecond latency and potentially unlimited throughput (e.g., gaming, IoT).
2. **Unstructured Data:** You are storing data streams, logs, or social media feeds where the format changes constantly.
3. **Rapid Prototyping:** You are building a startup app and the data model is evolving daily.